



Faculté des Sciences
Département d'Informatique
Kénitra

A.U. 2025/2026

Structures de Données Linéaires: Listes, Piles et Files

Licence MATHÉMATIQUES-MIP - S4

Pr. N. ENNEYA

1. Introduction

Dans ce chapitre on va étudier les structures de données linéaires :

- Les listes linéaires:
 - Tableaux,
 - Simplement chaînées
 - Doublement chaînées ;
- Les piles (LIFO);
- Les files (FIFO).



Université Ibn Tofaïl
Faculté des Sciences
Kénitra

A.U. 2025/2026

I. Les Listes

Licence MATHÉMATIQUES-MIP - S4

Pr. N. ENNEYA

1. Définition

Une liste est une structure linéaire utilisée souvent dans la programmation.

Une liste linéaire **L** sur un ensemble **E** est une suite finie, éventuellement vide, d'éléments de **E** repérés selon leur rang dans la liste **L**=(**e**₀, ..., **e**_{n-1}) . Cette suite possède les propriétés suivantes :

- L'existence d'un premier élément qu'on appelle **Tête** de la liste
- L'existence d'un dernier élément que nous appelons **Queue** de la liste ;
- L'existence pour tout élément, à l'exception de la queue, d'un **Successeur** ;
- L'existence pour tout élément, à l'exception de la tête, d'un **prédécesseur**.

Longueur d'une liste:

- Le nombre **n** d'éléments de **L** est appelé la **longueur** de **L**. Si le nombre est **zéro**, on dit que **la liste est vide**.

1. Définition

Position d'un élément :

Chaque élément de la liste est associé une position ou un rang.
Si $L=(e_1, \dots, e_n)$ est une liste et $n \geq 1$, alors on dit que :

- e_1 est le **premier (tête)** élément de la liste ;
- e_n étant le **dernier (queue)** élément de la liste ;
- e_i est l'élément de **position i** ou de **rang i**;
- e_i est l'élément **suivant (successeur)** de e_{i-1} , ;
- e_i est l'élément **précédent (prédécesseur)** de e_{i+1} ,.

Une position contenant l'élément e est une **occurrence** de e . Il est possible qu'un même élément apparaisse à plusieurs positions.

2. Spécification opérationnelle

Une liste L peut être implémentée en mémoire par deux façons:

- Un tableau où les éléments sont contigus.
- Une liste chaînée formée par des maillons (cellules) chaînés.

● A. Implémentation par tableau:

Une manière courante d'implémenter une liste est d'utiliser un tableau pour stocker les éléments dans des emplacements contigus.

Dans ce cas il faut prévoir une taille maximum pour le tableau et beaucoup d'espace risque d'être inutilisé.

Ce type d'implémentation n'est pas pratique à utiliser pour insérer ou supprimer des éléments dans la liste. En effet, il faut à chaque fois décaler tous les éléments du tableau.

Cependant les éléments sont accessibles directement par leur indice.

B. Implémentation par liste chaînée:

I. Listes simplement chaînées:

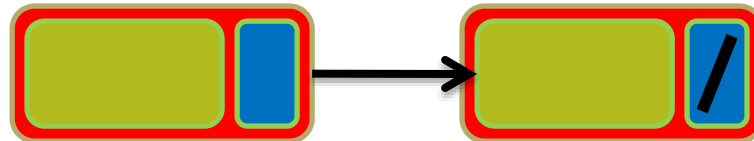
Définitions:

Notion de maillon : L'élément de base d'une liste chaînée s'appelle le maillon (ou cellule). Il est constitué :

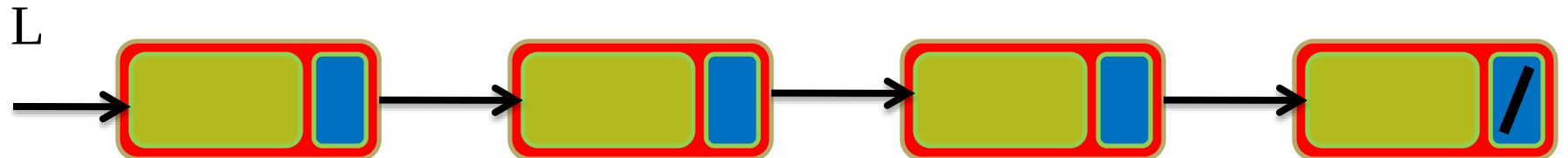
- d'un champ de données ;
- d'un pointeur vers un autre maillon.



Maillon suivant : Le champ pointeur d'un maillon pointe vers le maillon suivant de la liste. S'il n'y a pas de maillon suivant, le pointeur vaut **NULL**.



Notion de liste : Une liste simplement chaînée est un pointeur sur un maillon. Une liste vide est une liste qui ne contient pas de maillon. Elle a donc la valeur **NULL**.



B. Implémentation par liste chaînée:

I. Listes simplement chaînées:

Contrairement à la modélisation par tableau, l'espace mémoire utilisé par une liste chaînée n'est pas contigu. La taille d'une liste chaînée est inconnue à priori ; elle peut contenir autant d'éléments que l'on veut.

Une façon de modéliser une liste chaînée consiste à allouer dynamiquement les maillons (appelé aussi nœud ou cellule) chaque fois que cela est nécessaire. Dans ce cas, les maillons ne sont pas forcément contigus dans la mémoire

Pour assurer le lien entre les éléments de la liste, chaque élément contient en plus de la donnée un lien (pointeur vers l'élément suivant dans le cas d'une liste simplement chaînée).

La caractéristique fondamentale des listes chaînées est que les opérations d'insertion et de suppression ne nécessitent pas un décalage des éléments contrairement au cas des tableaux.

Implémentation:

On suppose avoir déjà déclaré le type **Telement**.

La déclaration des types constituant le maillon (l'élément de base d'une liste chaînée) est la suivante :

```
typedef int Telement;  
  
struct maillon {  
    Telement info;  
    struct maillon *suiv;  
};  
  
typedef struct maillon liste;
```

L'initialisation :

```
liste * listeVide() {  
    return NULL;  
}
```

Parcours d'une liste:

La longueur d'une liste:

Itérative:

```
int longueur(liste *L) {  
    int n=0;  
    liste *p=L;  
    while (p!=NULL) {  
        n++;  
        p = p->suiv;  
    }  
    return n;  
}
```

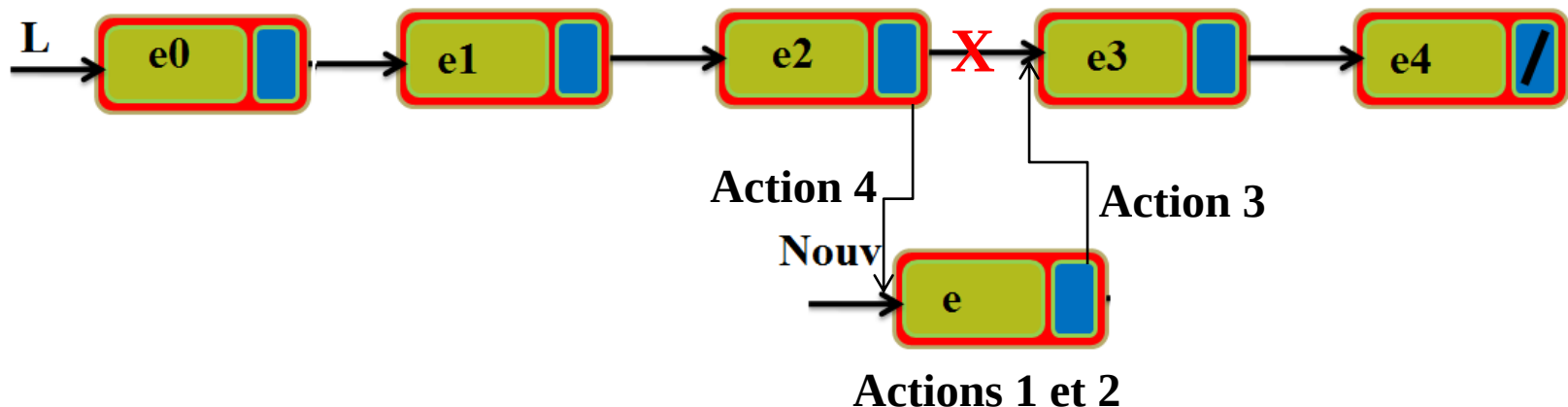
Réursive:

```
int longueurR(liste *L) {  
    if (L==NULL) {  
        return 0;  
    } else {  
        return 1 +  
            longueurR(L->suiv);  
    }  
}
```

Insertion :

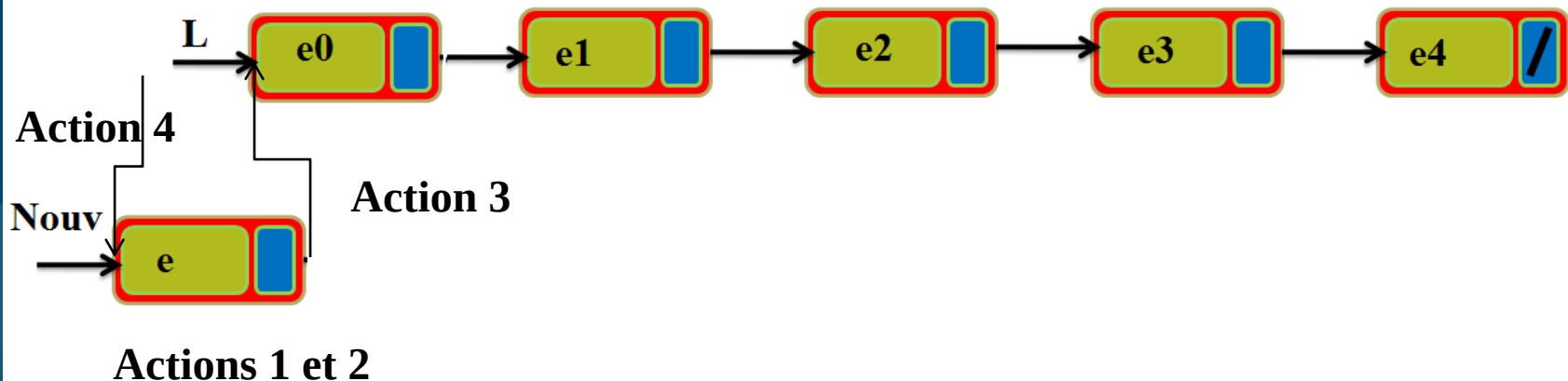
Les actions à exécuter pour insérer un élément sont :

1. Créer un nouveau maillon qui va contenir l'élément à insérer;
2. Mettre la donnée (l'élément à insérer) dans le nouveau maillon;
3. Rechercher la place d'insertion et mettre le pointeur suivant de nouveau maillon sur le maillon qui se trouve à la place du rang k dans la liste;
4. Mettre le pointeur suivant de maillon qui se trouve à la place rang $k-1$ sur le nouveau maillon.



Insertion :

Remarque : Pour le cas **d'insertion au début**, l'action 4 consiste seulement à modifier la tête de la liste (l'identificateur de la liste) en mettant le pointeur L sur le nouveau maillon créé.



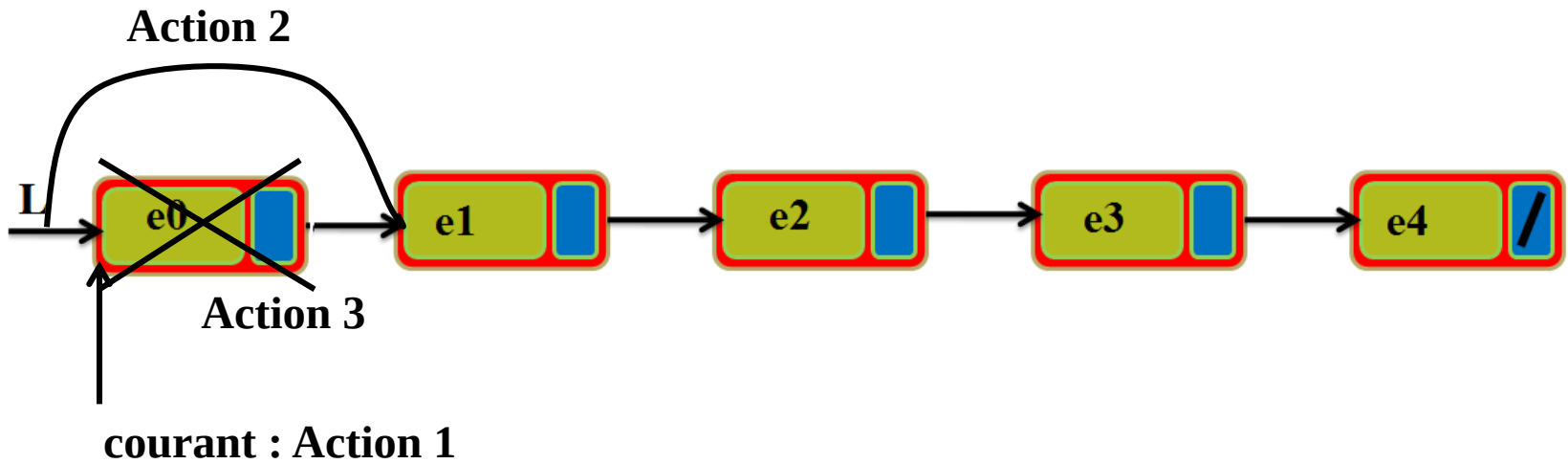
Insertion :

```
liste* inserer (liste *L, int k ,Telement e){
    liste *nouv, *courant;
    int i;
    if (k>=0 && k <=longueur(L)) {
        nouv=(liste*)malloc(sizeof(liste)); //Action 1
        nouv->info=e; // Action 2
        if (k==0) { //insertion au début
            nouv->suiv=L;
            L=nouv;
        } else { // insertion
            i=0; courant=L;
            while (i<k-1) {
                courant=courant->suiv;
                i=i+1;
            }
            nouv->suiv = courant->suiv; // Action 3
            courant->suiv = nouv; // Action 4
        }
    }
    return L;
}
```

Suppression :

Pour le cas de la suppression du premier élément, il y a trois actions, dans cet ordre, à réaliser :

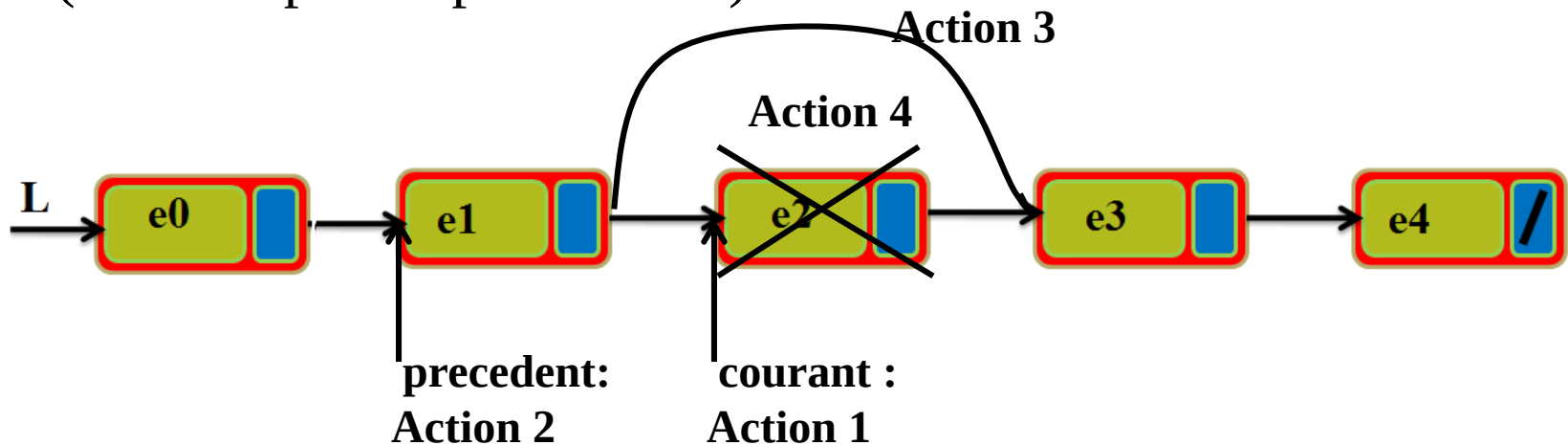
1. Faire pointer le premier élément par un pointeur auxiliaire
2. Faire pointer la tête de liste sur le deuxième élément de la liste,
3. libérer l'espace mémoire occupé par l'élément supprimé.



Suppression :

Pour le cas de la suppression d'un élément au milieu ou à la fin d'une liste, il y a quatre actions à exécuter dans cet ordre:

1. Mettre le pointeur courant sur l'élément à supprimer;
2. Mettre le pointeur précédant sur l'élément qui précède l'élément à supprimer;
3. Mettre le pointeur suivant du maillon pointé par précédant sur le suivant de maillon pointé par courant.
4. Libérer l'espace mémoire occupé par l'élément supprimé (élément pointé par courant).



Suppression :

```
liste* supprimer (liste *L, int k ){
    liste *prec, *courant;
    int i;
    if (k>=0 && k <longueur(L)){
        if (k==0){//suppression du premier Ã©lt
            courant=L;// Action 1
            L=courant->suiv; // Action 2
            free(courant); // Action 3
        }else{// Suppression des autres Ã©lts
            i=0; courant=L;
            while (i<k){//Actions 1 et 2
                prec = courant;
                courant=courant->suiv;
                i=i+1;
            }
            prec->suiv = courant->suiv;// Action 3
            free(courant); // Action 4
        }
    }
    return L;
}
```


L'accès à un élément (Fonction accéder):

Version itérative:

```
Telement acceder (liste *L, int k ){
    liste *courant;
    int i;
    if (k>=0 && k <longueur(L)) {
        i=0; courant=L;
        while (i<k){
            courant=courant->suiv;
            i=i+1;
        }
        return courant->info;
    }
}
```

Version récursive:

```
Telement accederR (liste *L, int k ){
    if (k==0)
        return L->info;
    else
        return accederR(L->suiv, k-1);
}
```

Recherche:

la fonction de recherche retourne -1 si l'élément rechercher n'existe pas dans la liste sinon elle retourne son rang dans la liste.

```
int rechercher (liste *L, Telement e ){
    liste *courant;
    int i;
    i=0; courant=L;
    while (courant != NULL) {
        if (egal(courant->info,e)) // fonction egal pour
            return i; // comparer deux enregistrements
        else{
            courant=courant->suiv;
            i=i+1;
        }
    }
    return -1;
}
```

Est Vide: La fonction qui vérifie si la liste est vide ou non.

```
int estVide(liste *L) {
    return L==NULL;
}
```

B. Implémentation par liste chaînée:

II. Listes doublement chaînées:

Définitions:

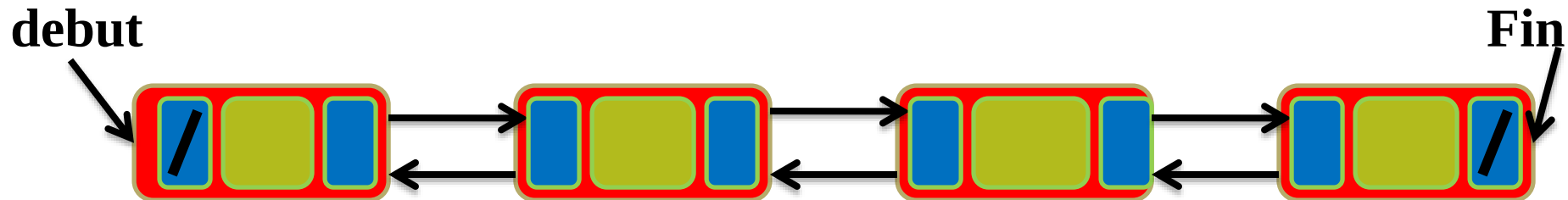
Une liste doublement chaînée est constituée par des maillons comportant trois champs :

- un champ de données info ;
- un pointeur vers l'éléments suivant de la liste;
- un pointeur vers l'éléments précédent de la liste;



Ces types de listes peuvent être parcourus dans les deux sens.

Les pointeurs précédent du premier élément et le suivant du dernier éléments ont la valeur **NULL**.

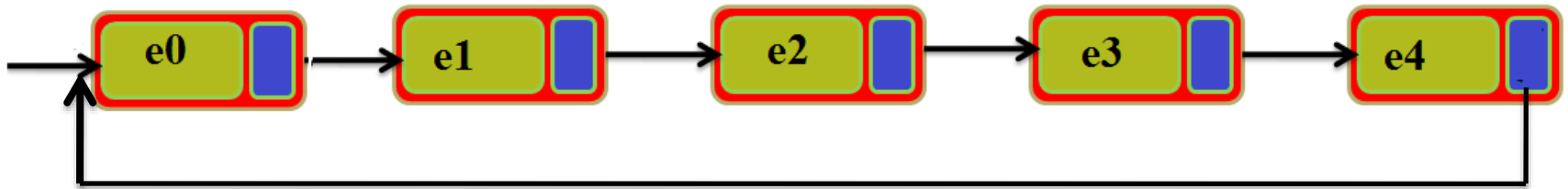


B. Implémentation par liste chaînée:

III. Listes circulaires :

Définition:

Une liste circulaire peut être simplement ou doublement chaînée. Sa particularité est de ne pas comporter de queue. Le dernier élément de la liste pointe vers le premier. Un élément possède donc toujours un suivant.



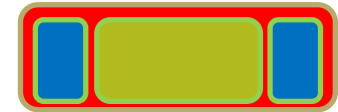
B. Implémentation par liste chaînée:

II. Listes doublement chaînées:

Définitions:

Une liste doublement chaînée est constituée par des maillons comportant trois champs :

- un champ de données info ;
- un pointeur vers l'éléments suivant de la liste;
- un pointeur vers l'éléments précédent de la liste;



Ces types de listes peuvent être parcourus dans les deux sens.

Les pointeurs précédent du premier élément et le suivant du dernier éléments ont la valeur **NULL**.

